SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – EXPERIMENTS

Course Code:	CSA0612	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	**CO3: Articulation** (BL4, BL5)	**Assessment:**	Assessment Tool 1 – Experiments
		Total Marks:	100
CO3 Statement:	Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication		
Student Name:	Shaik Azeem	**Reg. No.:**	192472307
Section / Batch:	CSE_AI	**Date:**	

Instructions to Students

- Answer the following question. Total Marks: 100.
- Write the algorithm and execute the code for the given experiment.
- Follow a well-structured, systematic approach to design and implement an optimal solution. • Provide insightful analysis with accurate validation, supported by clear documentation including diagrams, code, and results.

Question Type	Focus Area	Questions
Experiments	Evaluate divide and conquer matrix multiplication by implementing recursive splitting of matrices. Record execution time for different matrix sizes. Analyze how problem division reduces complexity. Interpret results and justify the efficiency of recursive approaches.	Q1

SECTION A – Experiments

Code of Conduct

I Shaik Azeem(192472307)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Shaik Azeem

RUBRICS FOR CALCULATION

Experiments					
Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**

Understanding of Concepts	25%	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30%	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20%	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15%	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10%	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%				
Experimental Design	30%				
Use of Tools	20%				
Analysis of Results	15%				
Documentation	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Evaluate divide and conquer matrix multiplication by implementing recursive splitting of matrices. Record execution time for different matrix sizes. Analyze how problem division reduces complexity. Interpret results and justify the efficiency of recursive approaches.

```python
import numpy as np
import time

# Divide and Conquer Matrix Multiplication
def divide_conquer(A, B):
    n = len(A)

    # Base case
    if n == 1:
        return np.array([[A[0][0] * B[0][0]]])

    mid = n // 2

    # Split matrices into 4 parts
    A11 = A[:mid, :mid]
    A12 = A[:mid, mid:]
    A21 = A[mid:, :mid]
    A22 = A[mid:, mid:]

    B11 = B[:mid, :mid]
    B12 = B[:mid, mid:]
    B21 = B[mid:, :mid]
    B22 = B[mid:, mid:]

    # Recursive multiplication
    C11 = divide_conquer(A11, B11) + divide_conquer(A12, B21)
    C12 = divide_conquer(A11, B12) + divide_conquer(A12, B22)
    C21 = divide_conquer(A21, B11) + divide_conquer(A22, B21)
    C22 = divide_conquer(A21, B12) + divide_conquer(A22, B22)

    # Combine sub-matrices
    top = np.hstack((C11, C12))
    bottom = np.hstack((C21, C22))

    return np.vstack((top, bottom))


# Test for different matrix sizes
sizes = [2, 4, 8, 16, 32, 64]
```

```
print("Divide and Conquer Matrix Multiplication")
print("-" * 50)

for n in sizes:
    # Generate random matrices
    A = np.random.randint(1, 10, (n, n))
    B = np.random.randint(1, 10, (n, n))

    start = time.perf_counter()

    C = divide_conquer(A, B)

    end = time.perf_counter()

    print(f"Matrix Size: {n} x {n}")
    print(f"Execution Time: {(end - start):.6f} seconds")
    print("-" * 50)

# Example output for 4x4 matrices
print("\nExample Multiplication (4x4)")
A = np.random.randint(1, 10, (4, 4))
B = np.random.randint(1, 10, (4, 4))
C = divide_conquer(A, B)

print("Matrix A:")
print(A)

print("\nMatrix B:")
print(B)

print("\nResult Matrix C:")
print(C)
```

```
PS C:\Users\shaik\Downloads\program\python\CSA0612> py test.py
Divide and Conquer Matrix Multiplication
--------------------------------------------------
Matrix Size: 2 x 2
Execution Time: 0.000226 seconds
--------------------------------------------------
Matrix Size: 4 x 4
Execution Time: 0.000302 seconds
--------------------------------------------------
Matrix Size: 8 x 8
Execution Time: 0.001800 seconds
--------------------------------------------------
Matrix Size: 16 x 16
Execution Time: 0.016010 seconds
--------------------------------------------------
Matrix Size: 32 x 32
Execution Time: 0.115543 seconds
--------------------------------------------------
Matrix Size: 64 x 64
Execution Time: 0.811057 seconds
--------------------------------------------------

Example Multiplication (4x4)
Matrix A:
[[7 7 8 7]
 [6 5 3 4]
 [6 4 1 8]
 [8 8 7 1]]

Matrix B:
[[6 5 4 2]
 [7 9 6 2]
 [8 5 5 6]
 [2 3 2 4]]

Result Matrix C:
[[169 159 124 104]
 [103 102  77  56]
 [ 88  95  69  58]
 [162 150 117  78]]
```

Report:

Input: Two square matrices **A** and **B** of size **n × n** (where *n* is a power of 2)
Output: Product matrix **C = A × B**

Formula

Divide matrix **A** into four submatrices:

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}$$

Divide matrix **B** into four submatrices:

$$B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

Compute the four submatrices of **C** recursively:

$$C_{11} = A_{11}B_{11} + A_{12}B_{21}$$

$$C_{12} = A_{11}B_{12} + A_{12}B_{22}$$

$$C_{21} = A_{21}B_{11} + A_{22}B_{21}$$

$$C_{22} = A_{21}B_{12} + A_{22}B_{22}$$

Combine the four submatrices:

$$C = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix}$$

Algorithm

1. **Start**
2. Read the input matrices **A** and **B** of size **n × n**.
3. **Base Case:** If **n = 1**, return
   C=A×BC=A \times BC=A×B
4. Divide **A** into **A11, A12, A21, A22**.
5. Divide **B** into **B11, B12, B21, B22**.
6. Recursively compute:
   - C11=A11B11+A12B21\mathbf{C_{11}=A_{11}B_{11}+A_{12}B_{21}}C11=A11B11+A12B21
   - C12=A11B12+A12B22\mathbf{C_{12}=A_{11}B_{12}+A_{12}B_{22}}C12=A11B12+A12B22
   - C21=A21B11+A22B21\mathbf{C_{21}=A_{21}B_{11}+A_{22}B_{21}}C21=A21B11+A22B21
   - C22=A21B12+A22B22\mathbf{C_{22}=A_{21}B_{12}+A_{22}B_{22}}C22=A21B12+A22B22
7. Merge **C11, C12, C21, C22** to form the final matrix **C**.

8. Record the execution time:
   Execution Time=End Time−Start Time\text{Execution Time} = \text{End Time} - \text{Start Time}Execution Time=End Time−Start Time
9. Repeat for different matrix sizes (e.g., **2×2, 4×4, 8×8, 16×16, ...**).
10. Compare the execution times and analyze the recursive approach.
11. Display the resulting matrix **C** and execution time.
12. **Stop.**

Time Complexity Formula

The divide-and-conquer recurrence is:

T(n)=8T(n2)+O(n2)T(n)=8T\left(\frac{n}{2}\right)+O(n^2)T(n)=8T(2n)+O(n2)

Using the Master Theorem,

T(n)=O(n3)T(n)=O(n^3)T(n)=O(n3)

Although the asymptotic complexity remains **O(n3)O(n^3)O(n3)**, recursive splitting improves modularity and serves as the foundation for faster algorithms such as **Strassen's Algorithm**.